# React Interview Prep

## Topic 14 — Code Splitting & Suspense: Complete Guide

**Analogy · Bundle Problem · React.lazy · Suspense · Split Strategy · Preloading · useTransition · useDeferredValue · Error Boundary · Q&A;**

| | |
|---|---|
| ■ Covers | All examples from conversation — nothing skipped |
| ■ Series | React Interview Prep — Session 14 |

by Akshay Chavhan • React Interview Series

## 1. Like You're 5 — Restaurant Meal

Imagine ordering a full meal — starter, main, dessert, drinks — but the waiter brings everything at once before you sit down. That's what happens without code splitting: all pages download upfront even if the user only visits the homepage.

> ■ ***Code splitting = bring the starter first. Main course downloads only when you're ready for it.***

## 2. The Problem — Bundle Size

**■ Wrong**

```
// ■ Without code splitting
// User visits homepage
// Browser downloads EVERYTHING:
//
// ■■■ HomePage.js (10kb) ← needed
// ■■■ AboutPage.js (8kb) ← NOT yet
// ■■■ DashboardPage.js (45kb) ← NOT yet
// ■■■ AdminPanel.js (60kb) ← NOT yet
// ■■■ ReportsPage.js (80kb) ← NOT yet
//
// Total first load: 203kb ■
```

**■ Correct**

```
// ■ With code splitting
// User visits homepage
// Browser downloads ONLY:
//
// ■■■ HomePage.js (10kb) ← needed ■
//
// User visits dashboard:
// ■■■ DashboardPage.js (45kb) ← now ■
//
// Total first load: 10kb ■
// Other pages load on demand!
```

> ■ ***Code splitting reduces initial bundle size → faster first load → better performance score.***

## 3. React.lazy — Dynamic Import

**■ Wrong**

```
// ■ Static import
// Loaded immediately — always in bundle
import DashboardPage
from './pages/DashboardPage';

// Even if user never visits dashboard
// its code is downloaded on first load
```

**■ Correct**

```
// ■ Lazy import
// Loads ONLY when component first renders
const DashboardPage = lazy(
() =>
import('./pages/DashboardPage')
);
// dynamic import() returns a Promise
// React.lazy wraps it as a component
```

```
// How React.lazy works internally:
```

```
const DashboardPage = lazy(() => import('./pages/DashboardPage'));
// ↑
// function that returns a Promise to a module
// React.lazy makes it usable as JSX component
```

## 4. Suspense — The Loading Fallback

When a lazy component is loading (Promise pending) — something must show in the UI. That's Suspense.

```
import { lazy, Suspense } from 'react';
const DashboardPage = lazy(() => import('./pages/DashboardPage'));
const ProfilePage = lazy(() => import('./pages/ProfilePage'));
function App() {
return (
<Suspense fallback={<div>Loading...</div>}>
{/* ↑ shown while lazy component is fetching */}
<Routes>
<Route path='/dashboard' element={<DashboardPage />} />
<Route path='/profile' element={<ProfilePage />} />
</Routes>
</Suspense>
);
}
```

### Better Fallbacks — Different Per Route

```
// ■ Ugly fallback
<Suspense fallback={<div>Loading...</div>}>
```

```
// ■ Spinner
<Suspense fallback={<Spinner />}>
```

```
// ■ Skeleton screen — best UX (content shape while loading)
<Suspense fallback={<PageSkeleton />}>
```

```
// ■ Different fallback per route
<Route path='/dashboard' element={
<Suspense fallback={<DashboardSkeleton />}>
<DashboardPage />
</Suspense>
} />
```

## 5. Where to Split — Strategy

| Split type | When to use | Impact |
|---|---|---|
| Route level (page) | Always — every page as separate chunk | Highest — biggest bundle reduction |
| Component level | Heavy components: charts, editors, video | Medium — reduces page-level bundle |
| Third-party library | Large libs: moment, lodash, chart.js | High — libraries are often huge |
| Don't split | Tiny components, shared utilities | Overhead > benefit — skip it |

### Component-Level Splitting — Heavy Components

```
// Heavy chart library — only load when needed
const HeavyChart = lazy(() => import('./components/HeavyChart'));
function Dashboard() {
const [showChart, setShowChart] = useState(false);
return (
<div>
<button onClick={() => setShowChart(true)}>
Show Analytics
</button>
{showChart && ( // only renders when true
<Suspense fallback={<ChartSkeleton />}>
<HeavyChart />
{/* Downloads ONLY when showChart = true ■ */}
</Suspense>
)}
</div>
);
}
```

## 6. Preloading — Load Before User Clicks

Trigger the import on hover — chunk downloads while user is deciding. By the time they click it is instant.

```
const AboutPage = lazy(() => import('./pages/AboutPage'));
function Navbar() {
function handleMouseEnter() {
import('./pages/AboutPage'); // trigger download on hover
// chunk starts downloading now — before user clicks
}
return (
<Link
to='/about'
onMouseEnter={handleMouseEnter} // preload on hover ■
>
About
</Link>
);
}
```

```
// User hovers → chunk downloads → user clicks → INSTANT ■
```

## 7. Error Boundary with Suspense — Handle Load Failures

If a lazy chunk fails to load (network error) — it throws. Suspense only handles pending state, not errors. Error Boundary catches failures.

```
class ErrorBoundary extends React.Component {
state = { hasError: false };
static getDerivedStateFromError() {
return { hasError: true };
}
render() {
if (this.state.hasError) {
return (
<div>
<p>Failed to load. Check your connection.</p>
<button onClick={() => this.setState({ hasError: false })}>
Retry
</button>
</div>
);
}
return this.props.children;
}
}
```

```
// Correct nesting — ErrorBoundary OUTSIDE Suspense
function App() {
return (
<ErrorBoundary> {/* catches load failures */}
<Suspense fallback={<Spinner />}> {/* shows while loading */}
<Routes>...
</Suspense>
</ErrorBoundary>
);
}
```

> ■ ***ErrorBoundary wraps Suspense — not the other way around. Suspense = pending state. ErrorBoundary = error state.***

## 8. useTransition — Non-Blocking Updates (React 18)

Mark a state update as non-urgent — keep old UI visible while new content loads instead of showing a blank skeleton.

```
import { useTransition, lazy, Suspense } from 'react';

function App() {
const [page, setPage] = useState('home');
const [isPending, startTransition] = useTransition();

function navigateTo(newPage) {
startTransition(() => {
setPage(newPage); // mark as non-urgent
});
}

return (
<div>
<button onClick={() => navigateTo('dashboard')}>
Dashboard
{isPending && <Spinner size='small' />}
{/* small spinner in button — NOT full page skeleton */}
</button>

<Suspense fallback={<PageSkeleton />}>
{page === 'home' && <HomePage />}
{page === 'dashboard' && <DashboardPage />}
</Suspense>
</div>
);
}
```

**■ Wrong**

```
// ■ Without useTransition
// Old page disappears immediately
// → shows skeleton
// → new page appears
// (jarring flash between pages)
```

**■ Correct**

```
// ■ With useTransition
// Old page STAYS visible
// + small spinner shows in button
// → new page appears smoothly
// (seamless transition ■)
```

## 9. useDeferredValue — Defer Expensive Renders

Let a value lag behind intentionally — keeps the UI responsive while expensive re-renders catch up.

```
import { useDeferredValue } from 'react';

function SearchResults({ query }) {
const deferredQuery = useDeferredValue(query);
// deferredQuery lags behind query intentionally

// Expensive list — only re-renders when React has spare time
const results = filterHugeList(deferredQuery);

return (
<div style={{ opacity: query !== deferredQuery ? 0.7 : 1 }}>
{/* dims while deferred — shows stale results vs blank ■ */}
{results.map(r => <ResultItem key={r.id} result={r} />)}
</div>
);
}
```

```
// User types fast:
// query → updates immediately → input stays responsive ■
// results → updates when React has time → no lag ■
```

> ■ ***useDeferredValue = 'this value can lag'. useTransition = 'this update is non-urgent'. Both keep UI responsive — different use cases.***

## 10. React 18 Concurrent Features — Summary

| Feature | What it does | Use when |
|---|---|---|
| React.lazy + Suspense | Splits code into chunks, shows fallback while loading | Route/component level code splitting |
| useTransition | Marks update non-urgent — keeps old UI visible | Page transitions, tab switching |
| useDeferredValue | Lets a value lag behind — prioritises urgent updates | Search filtering, expensive derived renders |
| Error Boundary | Catches errors in child tree including failed lazy loads | Always wrap Suspense in production |
| Preloading | Triggers import() on hover before user clicks | Routes user is likely to visit next |

■ ***All three React 18 features = better UX without manual loading state management. They work together — lazy + Suspense + useTransition = smooth navigation.***

## 11. Interview Q&A;

**Q** Beginner **Q1. What is code splitting?**

➜ Code splitting breaks your JavaScript bundle into smaller chunks that load on demand. Instead of downloading all code upfront, each chunk downloads only when needed. React.lazy with dynamic import() implements this — each lazy component becomes its own chunk that only downloads when first rendered.

**Q** Beginner **Q2. What is React.lazy and how does it work?**

➜ React.lazy takes a function that calls dynamic import() which returns a Promise resolving to a module. React wraps this in a lazy component — when the component first renders, React loads the chunk. Until it loads, the nearest Suspense boundary shows its fallback UI.

**Q** Beginner **Q3. What is Suspense?**

➜ Suspense is a component that catches a pending state from its children — either a lazy component still loading, or in React 18 a component waiting for async data. While pending, Suspense renders its fallback prop. When ready, it renders the actual children.

**Q** Intermediate **Q4. Where should you split code?**

➜ First at the route level — each page as a separate chunk is the biggest win with least effort. Then selectively at the component level for heavy components like charts or rich text editors. Don't split tiny components — the network request overhead outweighs the savings.

Q Intermediate

### Q5. What is preloading and when would you use it?

➜ Preloading triggers a dynamic import before the user clicks — typically on mouse hover. By the time the user clicks, the chunk is already downloaded making navigation feel instant. It is a UX optimisation for routes the user is likely to visit next.

Q Intermediate

### Q6. Why do you need an Error Boundary with Suspense?

➜ If a lazy component's chunk fails to download — network error, timeout — it throws an error. Suspense only handles the pending state, not errors. An Error Boundary wraps Suspense to catch load failures and show a retry UI instead of crashing the entire page.

Q Expert

### Q7. What is the difference between useTransition and useDeferredValue?

➜ Both are React 18 Concurrent features for keeping UI responsive. useTransition wraps a state update and marks it as non-urgent — React keeps the old UI visible while the new state resolves, showing a small pending indicator. useDeferredValue takes a value and lets it lag behind — useful when a derived computation is expensive and the input must stay immediately responsive.

Q Expert

### Q8. What is the difference between lazy loading and preloading?

➜ Lazy loading defers downloading a chunk until the component is first rendered — reduces initial bundle. Preloading downloads the chunk in advance while the user is hovering or before they navigate — makes the actual navigation feel instant. They complement each other: lazy = smaller first load, preload = instant subsequent navigation.